1.1 What is encapsulation? Explain how Java achieve it using access modifiers and getter/setter methods.

Ans:

Encapsulation is the process of wrapping data (variable) and methods into a single class and restricting direct access to the data.

Java achieves encapsulation mainly by:

- Declaring variables as `private`.
- Providing `public` getter methods to read data

Real life analogy:

A bank account is like a capsule. You cannot directly access the money inside the bank's system. You use controlled operations such as deposit and widthdraw.

1.2. What is Polymorphism ? Distinguish compile time vs runtime polymorphism with one example.

Ans:

Polymorphism means one method name can have different forms.

- Compile time polymorphism : Method overloading

Ex:

```
add (int a, int b)
add (int a, int b, int c)
```

- Run-time polymorphism : Method overriding.

Ex:

```
class Dog extends Animal {
        void sound() {
    System.out.println ("Bark");
    }
}
```

1.3. Write a BankAccount class demonstrating encapsulation (private balance + getBalance() / deposit()) and polymorphism. Include a main class

Ans: Code;

```
Class BankAccount {
    private double balance;

public BankAccount(douple balance) {
    this.balance = balance;
  }
  public void deposit (double amount){
    balance += amount;
  }
   public void deposit(double amount, String
                remarks) {balance += amount;
system.out.println("Remarks :" + remarks);
}}

   public class Main {
      public static void
      main(string[] args){
        account.deposit (500)
        account.deposit (1000, "Salary");
System.out.println(Balance: +account.getbalance());
```

Lab: 2 - Methods Overloading vs Overriding:

2.1: Compare Overloading vs Overriding: definition

Point	Overloading	Overriding
Definition	Same method name, different parameters	Subclass gives new implementation of parent method.
Class involved	Usually Same class	Parent + Child class.
Parameters	Must be different	Must be same.
Building time	Compile time (early binding)	Run time (late binding)

2.2 Early vs Late Binding

Early binding: Method call is decided at compile time. Ex: Overloading.

Late binding: Methods call is decided at run time. Ex: Overriding.

Overriding resolved at run time and overloading at compile time because:

- In overloading the compiler knows the method arguments, so it can be select the correct method before execution..
- In overriding, the actual object type may be different from the reference type, so Java decides which overridden method to call at run time.

L:2.3 Shape with area(), overridden by Circle and Rectangle binding via shape reference Add overloaded describe(string). Show output

Ans:

Shape, Circle and Rectangle - Java

Source code:

```
Class Shape {
    void area() {
     system.out.println("Shape area");
}
    void describe (string name) {
    system.out.println("shape:" +name);
```

```
}
    void describe(string name, int sides){
    system.out.println("Shape: "+name+",
                    sides: " + sides);
}
}

    class Rectangle extends Shape{
        @Override
        void area(){
        system.out.println("Area of
        Rectangle=" +(10*5)); } }

    public class Main{
    public static void main(String[] args){
        Shape s1 = new Circle();
        Shape s2 = new Rectangle();

    s1.describe("Cincle");
    s2. describe("Rectangle");
    }
}
```

LAB: 3| Abstract Class vs Interface Class

Difference between Abstract class and Interface are given below:

Abstract Class	Inferance Class
Represents às a relationship.	Represents can do capbility.
Can have normal fields.	Fields are public static final constants.
Can have constructors	Cannot have Constructors
Can have abstract + concrete methods.	Mainly have default methods
A class can extend only one class.	A class can implement multiple interfaces.

L:3.2| When should you choose an abstract class vs an interface.

Ans:

When we can choose

- Abstract class: Use when related classes share common code and properties.

  Example: Car is a vehicle.

- Interface; Use when different classes need a common ~~Ins~~ capability.

  Ex: Car can-do Insurable.

3.3| Source code:

```
abstract class Vehicle {
    void startEngine(){
    system.out.println("English started");
}
  abstract void fuelType();
}
    interface Insurable {
        void calculate premium();
```

```
Class Car extends Vehicle implements
        Insurance { void fuelType() {
    system.out.println ("Petrol");
  }
    public void calculate premium() {
     system.out.println("Premium : 5000");
  }
}

        Public class Main {
    public static void main(String[] args) {
      Car C = new Car();
        c.startEngine();
        c.fuel type();
        c.calculate premium();
    }
  }
```

LAB: 4] 4.1 Arraylist vs Vector Vs Linked list:

Feature	Array list	Vector	Linked List
Structure	Dynamic Array	Dynamic Array	Doubly linked list.
Synchroniza-tion	No	Yes	No
Random access	Fast	Fast	Slow
Insert/Delete	Slower in middle	Slower in middle	Fast at known Position.

4.2] Explain Set and its implementation.

Set: A Set is a collection that does not allow duplicate elements.

⇒ HashSet: No guaranteed order.

⇒ LinkedHashSet: Maintains insertion order

⇒ TreeSet: Treeset maintain sorted order. It maintains order using a balanced tree (Red-Black Tree).

4.3 Store 5 students in An ArrayList:

```
import Java.util.*;

public class Main{
   public static void
  main (string [] args) {

      Array list < string>
      students =new Array Lists <> ();
            students. add ("Wasi");
            students. add ("Rahim");
            students: add ("Ovi");

   system.out.println ("Array List:") ;
     for (string name: students)
   system.out.println ("Treeset:");
     for( string name: sorted)
       system.out.println(name);
   }
}
```

LAB5| 5.1: List ≥ 3 ways to implement multithreading in Java(extends, impelements, executon service)

Ans: ways to implement Multithreading:

1. Extends Thread:
   - Create a class that extends thread.
   - Override the `run()` method.

2. Implements Thread:
   - Create a class that implements Runable
   - Implement the run() method.

3. Executor service and callable:
   ⇒ Create a task that implements Callable and returns a result.
   ⇒ Submit the task to an ExecutorService

Preferred: Runable or executonService, because java supports single class interface and these approaches provides better flexibility.

5.2| Source code: Extends Thread:

```
class A extends Thread {
public void run() {
for(int i = 1; i <=5; i++)
system.out.println("Thread: "+ i);
}
}
class B implements Runable {
public void run() {
for(int i= 1; i <= 5; i++)
system.out.println("Runable :" + i); }
}
Public class Main {
public static void
main(string[] args)
A t1 = ne A();
Thread t2 = new Thread(newB());
t1.start();
t2.start();
}
}
```

5.3| Custom Exception :

```
Class InvalidRadius Exception
  extends Exception{
 InvalidRadius Exception(string msg){
            super(msg); }
  }
    class Cincle{
      double radius ;
 Cincle (double radius)
      if (radius <0)
       throw new
InvalidRadius Exception ("Radius cannot be Neg");
          this. radius = radius ; }
       double area() {
    return Math.PI * radius * radius;
}}
}
```

```
Public class Main {
    public static void main (string[]
        args ) { try
    {Cincle c = new Cincle (-5);
System.out.println("Area = " +c.area());
    }
}
```

LAB 6 JDBC with MySQL/Oracle (MVC pattern)

6.1 JDBC Steps:

1. Add MySQL JDBC driver.
2. Load/connect to database.
3. Create Connection.
4. Create PreparedStatement.
5. Execute SQL query.
6. Process ResultSet.
7. Close connection.

Important Classes:

- DriverManager
- Connection.
- PreparedStatement.
- ResultSet.

6.2 MVC Pattern.

MVC = Model + View + Controller.

- Model: Stores data, e.g. Student.
- View: Takes input and displays output.
  Ex: Main.
- Controller/DAO: Handles database operations. Ex: StudentDAO

6.3: Source Code:

```
import java.sql.*;
class Student {
    int id;
    string name;
    double cgpa;
```

```
Student(int id, string name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
}

    class StudentDAO {
     void insert(Student s)
    throws Exception {
     connection con = DiverManger.getConnection(
      preparationStatement PS = con.prepare (
    "Insert into Students Values(?,?,?)");
        PS.setInt(1, S.id);
        PS.setstring(2, S.name);
        PS.setDouble(3, S.cgpa);
    }
}

    Public class Main {Public static void main
    (string[] args) throws Exception {
     Student s = new student(1, wasi, 3.76);
System.out.println("student inserted");
```

LAB 7 JavaFX House Loan Calculator:

7.1 JavaFX Structure:

- Application: Main Javafx class.
- Stage : Main window
- Scence : Content area of the window.
- Grid Pane : Arranges components in rows and columns.
- VBOx : Arranges components vertically.
- Start() ; Main method where the Javafx UI is created.

7.2 & 7.3 code:

```
import Javafx.app.App;
import Javafx.geometry.Insets;
import Javafx.scene.Scene;
import Javafx.scene.control.*;
import Javafx.scene.layout.GridPane;
import Javafx.Stage.Stage.
```

```
Public class Loan calculator extend App {
    TextField loanField = new TextField();
    Text Field rateField = new Text Field();
    Text Field yearfield = new Text Field();
    Label monthly Label = new Text Label ("Monthly
                                Installment:");
    Label total Label = new Label ("Total Payment:");
    Label diff Label = new Label ("Difference:");

@Override
    Public void start (Stage stage) {
    GridPage grid = new GridPane();
    grid.setPadding(new Insects (10));
        grid.setHgap(10);
        grid.setVgap(10);
    grid.add(new Label ("Label Amount:"), 0, 0);
        grid.add(loan Field, 1, 0);
```

```
Button CalcBtn = new Button ("Calculate");
    grid.add (CalcBtn, 1, 3);
    grid.add (monthly label, 0, 4, 2, 1)
    grid.add (total Label, 0, 5, 2, 1);
    grid.add (diff label, 0, 6, 2, 1);
  CalcBtn.setOn Action (e → calculate());
stage.setScene (new scene (grid, 350, 300));
stage.settitle ("Loan Calculation");
Stage.show();
}
    void calculate() {
Double P = Double.parseDouble (loanfield.getText);
    int years = Integar.parseInt (yearsField);
      double r = annual Rate/12/100;
       int n = years * 12;
   double M = P * r * Math.Pow (1+r, n)/Pow (1+r,n)
Public static void main (string[] args) {
          launch (args); } }
```

LAB8 Socket Programming & RMI

8.1

Difference Socket and RMI

Socket	Java RMI
1. Low-Level communication.	1. High-level Communication.
2. Uses IP and Port.	2. Calls remote Java methods.
3. More control.	3. Easier for Java Objects
4. Good for chat/ network applications.	4. Good for distributed Java application.

8.2

Server Code:

```
import. Java.net.* ;
import. Java.io.* ;
public class Server{
public static void main(string[] args)'
        throws Exception {
serverSocket ss = new ServerSocket(5000
    Socket s = ss.accept ();
```

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(s.getInputSteam()
        printWriter Out = new PrintWriter
        (s.get output stream(), true);
    String msg = in.readLine();
    System.out.println("Client:" +msg);
    Out.println("Message received");
    s.close();
    ss.close();
  }
}
```

8.3 Client:

```
import Java.net.*;
import Java.io.*;
public class Client {
  public static void main(string [] args) throws
                                        Exception{
```

```
Socket s = new Socket ("localhost", 5000);
BuffenedReader in =
    new PrintWriter(s.getOutputStream(),
                    true);
    Out.println("Hellow Server");
    System.out.println("Server:" + in.readline());
        s.close();
    }
}
```

LAB:09 Servlet + JSP + JDBC

9.1: Steps:

1. Create database Student_db.
2. Create students table.
3. Add MySQL JDBC driver.
4. Create JSP Form.
5. Create Servlet.
6. Connect Servlet to MySQL using JDBC

L9.2 : Servlet Code:

```
Protected void doPost(HttpServletRequest
    req, HttpServletResponse res)
throws ServletException, IOException{
        int id= Integar.parseInt.
    string name=req.getParameter("name");
double cgpa =Double.parseDouble(req.
  getParameter("cgpa"));
try {
        connection con = DriverManager.
getconnection("Jdbc:mysql://localhost:
  3306/student_dP","root","password");
PS.setInt(1,id);
PS.setString(2, name);
PS, SetDouble(3, cgpa);
PS.executeupdate();
} catch (Exception) {
res.getWriter().println("Enron:"));
}
}
```

L9.3| JSP Display:

```
<%@ Page import = "Java.sql.*" %>

    <table border = "1">
    <tr>
     <th> ID</th>
     <th> Name</th>
     <th> CGPA</th>
  </th>
 <%
 Connection con = DriverManager.getConnection(

"Jdbc:mysql://localhost:3306/student_db",
        "root","password");

      statement st =
```